



SERVICIO NACIONAL DE APRENDIZAJE – SENA
CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL – CTPI

PROGRAMA Y FICHA DE FORMACIÓN
ADSO 3534105

TEMA

Soluciones Lógicas para la Vida Real y el Desarrollo de Software

PRESENTADO A:
ING.ANDRES MUÑOZ

PRESENTADO POR:
CARLOS DANIEL CUELLAR ANTURY
DANIER ANDRES HURTADO CAICEDO

Popayán – Cauca
29 agosto 2026

introduccion

Como estudiantes de Análisis y Desarrollo de Software del SENA, sabemos que la programación no comienza frente al teclado, sino en nuestra capacidad de razonar. Entender la lógica de un proceso antes de intentar escribir código es el paso más estratégico que podemos dar. Los algoritmos funcionan como el puente esencial entre el pensamiento humano y la ejecución tecnológica; son la hoja de ruta que transforma un problema complejo en una serie de instrucciones que una máquina puede ejecutar con precisión. Dominar esta estructura nos permite ser arquitectos de soluciones, no solo transcritores de sintaxis.

1. Parte 1: Investigación y Fundamentos de los Algoritmos

1.1 Definición y Utilidad de los Algoritmos en el Software

Basándonos en los conceptos de Khan Academy, podemos definir un algoritmo de forma sencilla: es un conjunto de pasos organizados para realizar una tarea específica. En el desarrollo de software, el algoritmo es el "plano" de la solución.

Evaluar y diseñar este plano antes de programar es crítico porque nos permite:

- **Ahorrar esfuerzo:** Detectamos errores de lógica en el papel, donde es fácil corregirlos, en lugar de buscarlos en miles de líneas de código.
- **Mejorar la velocidad:** Un algoritmo bien diseñado utiliza el camino más eficiente, lo que se traduce en programas que responden más rápido y consumen menos recursos del computador.

En nuestra cotidianidad, aplicamos esta lógica constantemente:

- **Registro en una aplicación:** El sistema solicita datos, valida que el correo tenga el formato correcto, verifica si el usuario ya existe y, solo si todo es válido, crea la cuenta y envía un mensaje de éxito.
- **Seguir una receta de cocina:** Se alistan los ingredientes, se procesan en un orden estrictamente lógico (secuenciación) y se ajusta el tiempo de cocción según el tipo de horno (selección) para obtener el plato final.
- **Retiro de dinero en cajero:** El algoritmo valida el PIN, verifica si hay fondos suficientes antes de entregar el efectivo (una validación de seguridad esencial) y finalmente actualiza el saldo del usuario.

Conexión narrativa: Tras entender que los algoritmos están en todas partes, es momento de analizar cómo se aplican en herramientas masivas que usamos a diario.

1.2 Análisis de una Aplicación Real: El Caso de los Domicilios (Rappi)

Plataformas como Rappi son ecosistemas complejos que resuelven problemas críticos de logística y tiempo mediante el uso inteligente de bloques lógicos.

- **Contextualización del problema:** Resuelve la dificultad de conectar la oferta de miles de comercios con la demanda de usuarios y la disponibilidad de repartidores, optimizando tiempos de entrega.
- **Actores e Interacción:** El sistema captura **entradas** vitales como la ubicación GPS del usuario, el pedido del menú y el método de pago.
- **Procesamiento y Resultados:** La aplicación ejecuta algoritmos de búsqueda de rutas y optimización. Un punto clave es el uso del bloque de **Selección**: el algoritmo debe decidir qué repartidor es el ideal basándose en la proximidad y disponibilidad actual. El **resultado final** es el producto entregado y un reporte digital de la transacción.
- **Evaluación de Impacto:** La automatización elimina los errores humanos del pedido telefónico tradicional (como direcciones mal anotadas) y reduce drásticamente el tiempo de espera, facilitando la toma de decisiones para el negocio.

Conexión narrativa: Este análisis nos permite ver que, aunque existan soluciones generales, a veces surge la necesidad de crear herramientas personalizadas.

1.3 Creación de Software Propio vs. Aplicaciones Existentes

Existe una diferencia estratégica entre adaptarse a una herramienta comercial y construir una solución a medida. Mientras una aplicación estándar ofrece funciones generales para el público masivo, el software propio nos permite atacar nichos de necesidad específicos con total flexibilidad.

Como aprendices SENA, podríamos usar una app de finanzas genérica, pero un software propio nos permitiría crear categorías exactas para nuestra realidad, como "presupuesto para fotocopias", "transporte a la sede" o "ahorro para certificación", ajustando las alertas a nuestro calendario académico.

Característica	Aplicación Estándar	Software Propio
Flexibilidad	Funciones rígidas y predefinidas.	Personalización total según el usuario.
Propuesta de valor	Solución generalista para muchos.	Solución específica para un problema puntual.

Conexión narrativa: Con esta base teórica clara, pasamos ahora a la fase práctica para diseñar una solución lógica a un problema detectado.

2. Parte 2: Reto - Diseño de la Solución (Control de Gastos Personales)

Para un estudiante, una gestión ineficiente de los recursos diarios no es solo un detalle administrativo; es un problema que afecta su estabilidad. No saber cuánto queda para el

pasaje del viernes o el almuerzo del martes genera un estrés innecesario que impacta directamente en su calidad de vida y enfoque en el estudio.

2.1 Descripción de la Situación Problemática

Actualmente, muchos compañeros llevan sus cuentas de forma manual o mental, lo que facilita la pérdida de información y errores en el cálculo del saldo real. Esta desorganización financiera afecta principalmente a quienes deben administrar presupuestos ajustados para materiales y transporte. Un programa digital es necesario para centralizar estos datos, evitar olvidos y permitir una planificación financiera real.

2.2 Propuesta de Solución mediante Software

La solución es un sistema de **Control de Gastos Estudiantiles**. El programa facilitará el registro inmediato de cada movimiento de dinero. Su funcionalidad principal será actuar como un asesor financiero personal que, además de guardar datos, ayude al estudiante a no gastar lo que no tiene, mejorando el ahorro y la organización del tiempo al automatizar los cálculos mensuales.

2.3 Estructura del Algoritmo (Entradas, Proceso y Salidas)

Para garantizar la robustez técnica, hemos estructurado la lógica siguiendo el modelo de "pre-condición" visto en el algoritmo de "Comprar un videojuego" de la fuente de Khan Academy: primero validamos, luego ejecutamos.

Entradas:

- Monto del movimiento (Número).
- Categoría (Alimentación, Transporte, Fotocopias, etc.).
- Tipo de operación (Ingreso o Gasto).
- Saldo actual almacenado en el sistema.

Lógica del Sistema:

- **Iteración:** Repetir el proceso mientras el usuario desee ingresar nuevos movimientos (bucle de entrada de datos).
- Recibir Monto, Categoría y Tipo de operación.
- **Selección:** Si el tipo es "Gasto": a. Verificar si el Monto $\leq$ Saldo Actual (Siguiendo la lógica de "Comprar un videojuego": si el dinero no es suficiente, no se puede realizar la acción). b. Si es suficiente: **Secuenciación** -> Restar Monto al Saldo Actual y almacenar en la categoría correspondiente. c. Si no es suficiente: Generar alerta de "Fondos insuficientes para este gasto".
- **Selección:** Si el tipo es "Ingreso": a. **Secuenciación** -> Sumar Monto al Saldo Actual.
- **Secuenciación:** Actualizar el historial general de transacciones.
- Preguntar al usuario si desea registrar otro movimiento (Control de la iteración).

Salidas:

- Confirmación de registro exitoso.
- Saldo total disponible actualizado.
- Reporte visual de gastos acumulados por categoría (ej. Total gastado en "Transporte").

El diseño de esta solución demuestra que el pensamiento algorítmico es la herramienta más poderosa para transformar problemas en resultados medibles.

3. Conclusiones y Referencias

El aprendizaje de los algoritmos nos otorga la capacidad de diseccionar cualquier desafío en sus bloques de construcción fundamentales: **secuenciación** para el orden lógico, **selección** para la toma de decisiones inteligentes e **iteración** para el procesamiento eficiente de grandes volúmenes de datos. Esta estructura mental no solo facilita la resolución de problemas en computación, sino que nos permite abordar situaciones de la vida real con una claridad y efectividad superiores.

3.1 Referencias Bibliográficas

- Khan Academy. (s.f.). *Introducción a los algoritmos*. Ciencias de la computación.
- Khan Academy. (s.f.). *Los bloques de construcción de algoritmos*. AP Computer Science Principles.